

モデルデータ変換ツール 作品説明書

1. プロジェクト基本情報

項目	内容
プロジェクト名	モデルデータ変換ツール
プロジェクト概要	既存ゲームの独自形式モデルデータをFBXへ変換し、Unreal Engineへインポートするためのエディタツール
開発環境	Unreal Engine
使用言語	C++、C# (Unreal Build Tool)
主な技術	Unreal Engine Editor API、Autodesk FBX SDK
バージョン管理	Perforce

2. 開発背景・目的・担当範囲

2.1 開発背景

顧客から、既存ゲームで使用していたモデルデータファイルをUnreal Engineへ導入したいという要望がありました。しかし、この独自モデル形式はUnreal Engineが標準で読み込める形式ではなく、そのままではモデルアセットとして利用できませんでした。

さらに、この形式を生成した元システムのソースコードは提供されておらず、利用できる正式な仕様も十分ではありませんでした。そのため、ファイル形式の調査から始め、モデルデータを復元した上で、Unreal Engineが扱える形式へ変換するツールが必要でした。

2.2 開発目的

- 独自モデル形式のバイナリ構造を調査し、モデルデータを解析できるようにします
- 頂点、法線、UV、頂点カラー、面、Transformなどを変換可能な中間データへ復元します
- 独自モデル形式のモデル情報をFBXのデータ構造へ正しく対応付けます
- 生成したFBXをUnreal Engineの既存インポート機能へ渡し、モデルアセットとして利用できるようにします
- 必要に応じて、生成したFBXをローカルで確認・修正してから再インポートできるようにします

2.3 個人の担当範囲

私の担当は、独自モデル形式からFBXへの変換機能です。具体的には、次の工程を担当しました。

- 公開情報と実ファイルを用いた独自モデル形式の調査
- バイナリReaderとモデル中間データの設計・実装
- 独自モデル形式からFBX向け中間データへの属性変換
- FBX SDKを用いたScene、Node、Mesh、Materialの生成

- FBXファイルの出力
- Unreal Engineへのインポートまでを接続する変換フローの構成
- 変換失敗箇所を確認するためのログ出力

3. 技術調査と方式選定

3.1 変換方式の比較

Unreal Engineへ独自形式のモデルデータを導入する方法として、主に二つの方式を検討しました。

方式	利点	課題
独自モデル形式をUnreal Engineへ直接インポートする	中間ファイルが不要	Unreal Engine側に独自形式の解析、Mesh生成、Material接続、エラー処理を個別実装する必要がある
独自モデル形式をFBXへ変換して既存のFBXインポーターを利用する	標準的な中間形式を利用でき、既存のインポート処理とチーム内の拡張機能を再利用できる	独自モデル形式とFBXのデータ表現の差を正しく変換する必要がある

3.2 FBXを中間形式に選んだ理由

最終的に、独自モデル形式をFBXへ変換してからUnreal Engineへインポートする方式を選択しました。理由は次の通りです。

1. Unreal Engineの既存FBXインポート機能を利用できます
2. 同じチームで進めていたFBXインポート拡張と連携できます
3. 独自Importerにすべてのアセット生成処理を重複実装する必要がありません
4. FBXを中間成果物としてローカルに保存し、変換結果を確認できます
5. 必要な場合はDCCツールでFBXを修正してから再インポートできます
6. 独自モデル形式の解析とUnreal Engine側のインポート処理を疎結合にできます

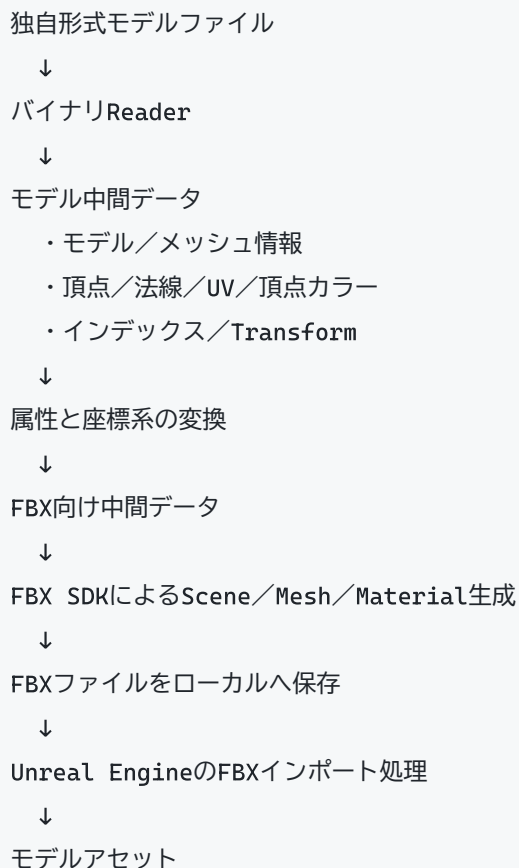
この方式により、私は独自モデル形式に固有の解析とFBXへの意味変換に集中し、Unreal Engine側では実績のあるインポート経路を活用できました。

3.3 FBX SDKを採用した理由

FBXには、頂点だけでなく、Scene Graph、Mesh、Node、法線、UV、頂点カラー、Material、Texture参照、Transformなどの構造があります。これらを独自にシリアライズすると、FBX仕様への追従と互換性確認に多くの実装が必要になります。

Autodesk FBX SDKを利用することで、FBXの意味構造をAPIとして明示的に組み立てられます。そのため、ファイル形式自体の書き出し処理ではなく、独自モデル形式の各属性をFBXのどの要素へ対応付けるかという本プロジェクト固有の課題に注力できました。

4. 全体の処理構成



入力形式に依存する独自モデル形式の解析と、出力形式に依存するFBX生成の間に中間データを置きました。これにより、バイナリ解析、属性変換、FBX生成を別々に確認できるようにしました。

5. 代表的な技術課題と解決方法

5.1 正式な仕様が不十分な独自モデル形式の調査

課題

開発開始時点では、独自モデル形式を生成した元システムのソースコードを利用できませんでした。また、この形式には複数のversionが存在し、versionごとにデータ構造が異なっていました。

調査と判断

まず、インターネット上で参照可能な情報を収集し、ファイル内に含まれる可能性があるデータと配置を整理しました。次に、複数versionの実ファイルを十六進数表示で確認し、各区画の先頭に現れる特徴的な文字列を手掛かりとしてデータ位置を特定しました。その上でversion値を読み取り、既知のモデル情報と後続のバイト列を比較しながら、versionごとのフィールド構成と読み取り順を確認しました。

実施内容

一度の推測で仕様を決めるのではなく、次の手順を繰り返しました。

公開情報から暫定的な構造を整理

↓

十六進数表示から特徴的な文字列を探索

↓

versionごとのデータ構造を整理

↓

Readerへ分岐を実装

↓

既知のモデル情報と変換結果で再検証

結果

この過程を通して、独自形式のモデルデータを変換に必要な粒度で復元できるようにしました。

6. 成果

6.1 達成内容

- ・ ソースコードと十分な正式仕様がないう状態から、変換に必要な独自モデル形式の構造を調査・特定しました
- ・ 独自モデル形式を解析し、FBXとして出力する一連のツールを実装しました
- ・ 対象となった独自形式ファイルをFBXへ変換し、Unreal Engineへインポートしました
- ・ Unreal Engineの既存FBXインポート機能とチーム内の拡張機能を再利用しました
- ・ FBXを中間成果物として保存し、確認・修正・再インポートできる処理経路を用意しました

6.2 変換結果の確認

変換後のモデルについては、クライアントが実際のソフトウェアへ順次導入し、作業を進めながら表示結果を確認しました。加えて、開発側では提供されたサンプルの一部を対象に、変換後のモデルが正しく表示されることを確認しました。このように、実運用上の確認とサンプルを用いた抜き取り確認を組み合わせながら、変換処理の妥当性を検証しました。

7. 本プロジェクトで得た知識・能力

7.1 バイナリフォーマットを調査する能力

不完全な公開情報と実ファイルを組み合わせ、十六進数表示内の特徴的な文字列からデータ位置を特定し、versionごとの差分を比較してデータ構造を整理する能力を身につけました。単に値を推測するのではなく、検証可能な根拠を段階的に積み上げる重要性を学びました。

7.2 FBXのデータ構造に関する理解

FBX SDKを通して、Scene Graph、Node、Mesh、Control Point、Polygon、Layer Element、Material、Texture、Transformの関係を学びました。モデルを単なる頂点と面の集合ではなく、属性と階層を持つデータとして扱う理解が深まりました。

8. 技術的課題と今後の改善

8.1 変換結果の検証方法の体系化

本プロジェクトでは、クライアントが変換後のモデルを実際のソフトウェアへ導入する過程で表示結果を確認し、開発側でも提供されたサンプルを用いた抜き取り確認を行いました。

一方で、この方法は人の目による確認が中心となるため、対象データが増えるほど確認に必要な工数が大きくなり、細かな表示上の差異を見落とす可能性もあります。

今後は、画像認識とAI技術を組み合わせ、モデル変換後の表示結果から不具合の候補を自動検出するデバッグ支援機能を導入したいと考えています。固定したカメラ、照明、描画条件の下で取得した画像をAIに解析させることで、形状の崩れ、MaterialやTextureの表示異常などを検出・分類し、問題が疑われるモデルと確認すべき箇所を提示します。これにより、目視確認を中心としたデバッグ作業の一部を自動化し、確認工数の削減と不具合の見落とし防止につながれると考えています。